

SISTEMA DE VISIÓN ARTIFICIAL PARA DETECCIÓN DE OBSTÁCULOS Y PERSONAS

Implementación de Redes Neuronales en ESP32-P4 con ESP-DL

Simulación Emulada por SPIFFS y Visualización Serial en PC

Reporte Técnico de Proyecto

Curso: Inteligencia Artificial

Placa de Desarrollo: Guiton JC-ESP32P4-M13 DEV (ESP32-P4)

Cámara Auxiliar: OV5647 5MP V1.3 (Raspberry Pi)

Fecha de Generación: Mayo de 2026

1. Resumen Ejecutivo

Este reporte describe el diseño, desarrollo e implementación del proyecto de visión artificial orientado a la detección de obstáculos y personas en un vehículo de reparto (delivery). El proyecto integra hardware embebido de alto rendimiento (ESP32-P4), la biblioteca oficial ESP-DL de Espressif para inferencia de redes neuronales, y un pipeline de adquisición de datos basado en MATLAB y Roboflow.

Debido a limitaciones de software de los controladores de la cámara MIPI CSI en el ESP32-P4 dentro del SDK de Espressif al momento del desarrollo, se implementó una solución alternativa: el microcontrolador carga imágenes estáticas de alta resolución precargadas en el sistema de archivos SPIFFS, ejecuta la inferencia de la red convolucional a nivel local y transmite los resultados obtenidos (coordenadas de cajas delimitadoras) en formato JSON por puerto USB Serial a una PC. En la PC, un script en Python mapea estas coordenadas en tiempo real con las imágenes locales y las dibuja usando OpenCV.

2. Arquitectura de Hardware y Conectividad

El sistema físico está compuesto por:

- Microcontrolador Guiton JC-ESP32P4-M13 DEV:

Esta tarjeta incorpora el procesador SoC ESP32-P4, un chip RISC-V de doble núcleo a 400 MHz diseñado con extensiones de instrucciones matemáticas para aceleración de algoritmos de inteligencia artificial. Dispone de 32MB de PSRAM y 16MB de memoria Flash. También integra un coprocesador secundario ESP32-C6 para conectividad inalámbrica (WiFi/Bluetooth).

- Cámara Compatible con Raspberry Pi 5MP V1.3:

Sensor OV5647 con lente de enfoque fijo y cable plano de 15 pines. La placa de desarrollo cuenta con un conector físico FPC de paso fino que se conecta al bus MIPI CSI del microcontrolador.

A pesar de que los componentes están conectados físicamente, las limitaciones del SDK actual forzaron la emulación de la captura a través del sistema de archivos flash debido a la complejidad de reescribir los controladores del bus MIPI CSI y CCI desde cero en el firmware.

3. Limitaciones Técnicas y Solución de Contingencia

Durante las pruebas de integración del hardware, se observó que la biblioteca 'esp32-camera' no ofrecía soporte nativo completo para el bus MIPI CSI del ESP32-P4 con la cámara OV5647. Para lograr la captura de video en tiempo real, el equipo de desarrollo se enfrentó a la necesidad de implementar un driver de bajo nivel personalizado para coordinar la transmisión de datos CSI y el bus de control CCI (I2C), lo cual era inviable bajo el cronograma asignado.

Como plan de contingencia, se desarrolló la emulación por software:

- Las imágenes de prueba originales del entorno real del carro se compilan e instalan en una partición SPIFFS de 1MB en la flash del ESP32.
- El bucle del firmware monta esta partición, lee cada JPEG secuencialmente, lo decodifica a RGB888 y ejecuta la inferencia neural.
- La salida serial en formato JSON transmite los resultados estructurados a la PC, simulando con éxito el pipeline de análisis de imágenes sin depender de la captura activa de video.

4. Pipeline del Dataset y Modelo de Inteligencia Artificial

El entrenamiento y despliegue del modelo de IA siguió las siguientes fases:

1. Adquisición y Preprocesamiento de Datos:

Se capturaron múltiples videos en formato MP4 (Delivery1.mp4 a Delivery6.mp4). Mediante el script en MATLAB 'vid2frames.m', se extrajeron los fotogramas clave y se redimensionaron a una resolución de 224x224 píxeles utilizando interpolación bilineal para acoplarse a la entrada del modelo.

2. Etiquetado y Configuración del Dataset:

Las imágenes redimensionadas se subieron a Roboflow, creando el dataset 'Delivery_Vision' (versión 4) para detectar dos clases: 'obstacle' (ID 0) y 'person' (ID 1). La configuración de clases está dada en el archivo 'data.yaml'.

3. Compilación del Modelo Embebido:

El modelo YOLOv8 entrenado se optimizó y quantizó a enteros de 8 bits (INT8) usando la biblioteca ESP-DL. Esto generó el modelo quantizado de alto rendimiento 'espdet_pico_224_224_obstacle_person.espdll' de 577 KB, el cual se cargó en la flash del ESP32-P4 para inferencias ultrarápidas aceleradas por hardware.

5. Pipeline de Inferencia Embebida y PC Viewer

El flujo operativo del sistema embebido y visualizador es el siguiente:

- Firmware Embebido (C++ / ESP-IDF):

El código en 'app_main.cpp' monta el sistema de archivos SPIFFS, lee los archivos .jpg de la carpeta '/spiffs', los decodifica de JPEG a RGB888, ejecuta la inferencia con el modelo cargado 'ESPDetect', y escribe en el puerto serial los resultados en JSON estructurado delimitados por los marcadores '>>>JSON_START<<<' y '>>>JSON_END<<<'.

- Visualizador PC (Python / OpenCV):

El script 'pc_viewer.py' lee en tiempo real la terminal serial (COM). Al recibir un JSON válido, carga la misma imagen desde el almacenamiento local de la PC, dibuja las cajas delimitadoras y escribe la etiqueta del objeto detectado ('obstacle' o 'person') junto con su confianza, desplegando la imagen con OpenCV de forma interactiva.

6. Gestión de Versiones con Git y GitHub

Para garantizar la portabilidad y limpieza del proyecto, se configuró un repositorio Git. El archivo '.gitignore' cumple un rol fundamental al evitar que archivos binarios pesados (videos MP4 de 200MB+, archivos ZIP grandes y miles de fotogramas redundantes) ensucien el repositorio. Solo se suben los scripts, configuraciones del dataset, código de firmware y las imágenes mínimas de prueba.

Pregunta Clave: ¿Los cambios en GitHub se reflejan automáticamente en la PC?

NO. Git y GitHub son sistemas descentralizados. No existe una sincronización automática en segundo plano como ocurre con OneDrive o Dropbox. La sincronización es manual y controlada:

1. De Local (PC) a Remoto (GitHub):

Tus cambios locales permanecen en tu computadora hasta que los agregues, hagas commit y los subas:

- 'git add .' (para agregar cambios)
- 'git commit -m "mensaje"' (para consolidar la versión local)
- 'git push origin main' (para enviar a la nube de GitHub)

2. De Remoto (GitHub) a Local (PC):

Si modificas un archivo desde la página web de GitHub, o si otra persona sube cambios al repositorio, estos NO se descargarán solos. Para actualizar tu PC local con la versión de la nube, debes abrir una terminal en tu proyecto y ejecutar el comando manual:

- 'git pull origin main'

Esta independencia previene que tu entorno de desarrollo local sea sobrescrito o dañado de manera accidental por cambios externos.